

Guía de entrega

Cómo pasarle el sistema a otra persona sin que se pierda nada en el camino: qué enseñarle primero, quién se queda con qué responsabilidad, cómo se pide un cambio y cómo se sabe que quedó bien. Está escrita para que la lean los dos —quien entrega y quien recibe— y queden de acuerdo en el mismo proceso.

Actualizada el 10 de septiembre de 2026

Este documento se genera desde `docs/guia_de_entrega.md`. No se edita a mano: se regenera con `lisart guia entrega`.

Contenido

Cómo leer esta guía

0. Tres cosas antes de entregar nada

1. Qué documentos enseñar, y en qué orden

2. Front-end y back-end, para quien no programa

3. Quién se queda con qué

4. El proceso, paso a paso

5. Cómo revisar el trabajo sin leer código

6. Las reglas que no se negocian

7. Lo que el desarrollador va a querer cambiar, y por qué no (todavía)

8. El primer encargo, y los que siguen

9. Palabras que se usan aquí

Cómo leer esta guía

Tiene dos lectores, y no leen lo mismo:

SI ERES…	LEE	PUEDES SALTARTE
Quien entrega (la escuela, sin saber programar)	0, 1, 2, 3, 4, 5 y 6	7 y 8
Quien recibe (el desarrollador)	1, 3, 4, 6, 7 y 8	2 (ya lo sabes) y 5

La sección **4** es el acuerdo entre los dos. Conviene leerla juntos, en la reunión de entrega, y que ninguno se lleve una versión distinta.

Los otros tres documentos del repo contestan preguntas distintas; esta guía no los repite, los ordena:

DOCUMENTO	CONTESTA
`documentacion_general.pdf`	**qué** se construyó y por qué
`guia_de_uso.pdf`	**cómo se usa** cada semana
`mapa_del_codigo.pdf`	**por dónde se lee** el código y qué se rompe si algo falla
esta guía	**cómo se pasa** de una persona a otra

0. Tres cosas antes de entregar nada

Estas tres son de quien entrega, no del desarrollador, y **van antes de mandar un solo archivo**. Dos son los pendientes A1 y A2 de la bitácora; entregar el proyecto es el momento en que dejan de poder esperar.

1. **Cambiar la llave de Jotform.** La llave con la que se bajaron los datos de 115 alumnos sigue escrita dentro del cuaderno viejo (`projects/data_mechanism/respuestas_inscripcion_forms.ipynb`). Entrar a Jotform, generar una llave nueva, ponerla en `.env` , borrar la vieja en Jotform y **borrar del cuaderno la línea que la contiene**. Mientras alguien más tenga esa llave, puede bajar los datos de los alumnos.
2. **Crear el historial (`git init` y primer commit).** Hoy el proyecto es una carpeta suelta. Sin historial, un borrado accidental se lleva todo y no hay forma de ver *qué cambió* el desarrollador ni de deshacerlo. El `.gitignore` ya está listo: deja fuera los datos, la llave y el entorno. Se hace **después** del punto anterior, porque lo que entra al historial se queda para siempre.
3. **Decidir qué datos se entregan.** `data/` contiene nombres, domicilios, teléfonos y firmas de **menores de edad**. No es parte del código.

Qué se entrega y qué no

SE ENTREGA	NO SE ENTREGA (O NO TODAVÍA)
El código (`src/` , `tests/` , `assets/` , `Makefile` , `pyproject.toml`)	`data/raw/` , `data/interim/` , `data/master/` salvo catálogos, `data/backups/`
La documentación completa (`docs/` , `README.md` , `CLAUDE.md` , `.claude/estado-del-proyecto.md`)	`.env` (la llave). El desarrollador llena su propio `.env` desde `.env.example`
Los catálogos: `espacios` , `instrumentos` , `conceptos` (no llevan datos personales)	`.venv/` (643 MB que se reinstalan en un minuto con `make install`)
El vault de Obsidian (`vault/` , son notas de estructura)	`data/exports/` y cualquier `.xlsx` (llevan datos de cobranza por persona)

Se entrega **el repositorio con historial**, no una copia de la carpeta: la copia se lleva toda la columna derecha sin que nadie lo decida.

Sobre los datos reales, la recomendación es por etapas:

1. **Primera semana: sin datos reales.** `lisart init` crea la base vacía, `lisart ingest catalogos` carga precios y descuentos desde el Excel de la escuela (no lleva datos personales) y `lisart demo` siembra profesores y clases de prueba. Con eso se instala, se leen los documentos, pasan las pruebas y se hace el primer cambio, sin un solo alumno de verdad.
2. **Cuando hagan falta datos reales** —la proyección y la deduplicación solo se entienden con datos—, se entregan **con un acuerdo de confidencialidad por escrito**, **nunca en una carpeta sincronizada** a la nube, y se borran al terminar el encargo. `LISART_DATA_DIR` en `.env` permite tenerlos fuera de la carpeta del código, justo para esto.

1. Qué documentos enseñar, y en qué orden

Hay bastante escrito; el error sería entregarlo todo junto. Este es el orden, con lo que cada documento contesta y cuánto tarda leerlo:

#	DOCUMENTO	CONTESTA	TIEMPO	CUÁNDO
1	`README.md`	Cómo se instala y qué comandos hay	15 min	Antes de la reunión
2	`docs/documentacion_general.pdf`, secciones **1, 3, 5, 12 y 14**	Qué es esto, cómo está armado, qué fue difícil y qué falta	1 h	Antes de la reunión
3	`docs/mapa_del_codigo.pdf`, completo	Por dónde empezar a leer, qué se rompe si cada archivo falla, qué es front-end y qué back-end	1 h	**En la reunión**, es el documento de entrega
4	`CLAUDE.md` y `.claude/estado-del-proyecto.md`	Las reglas del repo y la **bitácora**: cada decisión con su porqué, y los pendientes ordenados	1 h	Primer día de trabajo
5	`docs/guia_de_uso.pdf`	Cómo se opera cada semana: lo que hace la administración	45 min	Primera semana. Quien mantiene el sistema tiene que saber usarlo
6	`docs/convenciones_datos.md` y `docs/modelo_de_datos.md`	Las reglas de los datos y el detalle de cada tabla	consulta	Cuando toque tocar una tabla
7	`vault/` en Obsidian	El sistema como grafo	opcional	Cuando quiera ver el mapa completo

El documento central es el número 3. Si solo hay tiempo para uno, es ese: su sección 8 dice por dónde empezar a leer el código, su sección 9 qué se puede tocar con confianza y qué no, y su sección 12 son las cinco frases que hay que decirle a quien recibe el proyecto.

El documento que se va a usar más es el número 4. La bitácora (`.claude/estado-del-proyecto.md`) es donde está escrito *por qué* cada cosa es como es: por qué la base son archivos CSV, por qué no hay tabla de familias, por qué el porcentaje de descuento se guarda en puntos. Un desarrollador nuevo va a querer cambiar varias de esas decisiones en la primera semana; la bitácora es lo que le ahorra rehacer una discusión que ya se tuvo.

Lo que **no** hace falta el primer día: `docs/guion_demo.md` (es un documento de trabajo para una presentación pasada) y el cuaderno de `projects/` (es el notebook original que el sistema reemplazó; se conserva por historia).

2. Front-end y back-end, para quien no programa

La pregunta es razonable y la respuesta corta engaña. Primero lo general, luego lo que es distinto en este proyecto.

En general. Un sistema tiene dos mitades. El **back-end** es la parte que decide **qué es verdad**: qué datos existen, cómo se guardan, qué se puede escribir y qué no, cuánto cuesta una colegiatura. El **front-end** es la parte que decide **cómo se ve**: la pantalla, los botones, los colores, el dibujo. Una analogía que sirve: en un restaurante, el back-end es la cocina y la bodega —qué hay, cuánto cuesta, qué se guarda—, y el front-end es el menú y el comedor. Un cambio en el menú no cambia lo que hay en la bodega; un cambio en la bodega puede dejar el menú mintiendo.

En Lisart. No hay un front-end clásico: no hay una aplicación de navegador aparte que hable con un servidor. Todo es Python corriendo en la computadora de la escuela, y Streamlit le manda al navegador el resultado ya dibujado. Por eso las dos palabras se redefinen así, y así están en el mapa del código:

	QUÉ HACE AQUÍ	DÓNDE VIVE
Back-end	Define las 17 tablas; lee y escribe los CSV sin dejar ninguno a medias; valida tipos, llaves y referencias; deduplica personas; calcula la proyección y los descuentos; hace respaldos	`tables.py`, `io.py`, `validate.py`, `ids.py`, `backup.py`, `procesos/`, `fuentes/`
Front-end	Dibuja las dos pantallas, el grafo, el pentagrama y la semana; imprime lo que sale en la terminal; genera los PDF y el vault	`ui/`, `assets/estilo.css`, lo que imprime `cli.py`, `guia.py`, `vault.py`

Lo que hay que saber, y que el desarrollador va a confirmar en el mapa del código (secciones 4 y 6):

1. **La línea no es la carpeta `ui/`, es el archivo `io.py`.** Todo lo que escribe en `data/` es back-end, esté donde esté. Y hay tres funciones dentro de la pantalla que sí escriben datos. El mapa las nombra.
2. **La terminal también es front-end.** Lo que `lisart ingest` o `lisart proyeccion` imprimen lo lee una persona cada semana; es la interfaz que más se usa.
3. **Los dibujos están escritos como texto (SVG).** No es capricho: ninguna librería de gráficas funciona en este equipo, está comprobado. Un desarrollador nuevo va a querer "arreglarlo" instalando una; no funciona.

Por qué esto te importa aunque no programes: cuando pidas un cambio, conviene decir de cuál mitad es. *"Que la pestaña de alumnos muestre el teléfono más grande"* es front-end: bajo riesgo, se hace y se mira. *"Que los descuentos se apliquen uno tras otro en vez de sumarse"* es back-end: cambia cuánto se cobra, toca dinero y datos que ya existen, y necesita pruebas, respaldo y una nota en la bitácora. **El mismo tamaño de frase puede ser una tarde o una semana**, y la diferencia es en qué mitad cae.

3. Quién se queda con qué

Delegar el mantenimiento no es delegar el sistema. Esto es lo que **no** pasa al desarrollador, y hay que decirlo desde el primer día para que no lo asuma:

SE QUEDA CON LA ESCUELA	PASA AL DESARROLLADOR
Las llaves y las cuentas: Jotform, Kydemy, WhatsApp. Se dan de alta a nombre de la escuela y se comparten por `.env`, nunca por chat	El código, sus pruebas y su documentación
Los datos reales, y decidir quién los ve	Que `make check` pase antes de entregar
Los catálogos: espacios, instrumentos, conceptos, precios, descuentos. Los edita la administración; el código solo avisa	Regenerar los PDF, el vault y `modelo_de_datos.md` cuando cambie algo
Las decisiones de negocio: cómo se combinan los descuentos, qué cuenta como pago, qué mensaje se manda. El desarrollador implementa, pregunta y propone, pero no decide	Proponer, con argumentos, cuando una decisión de negocio encarece o complica el código
La prioridad: qué pendiente va primero. La lista está en la bitácora, sección 5	Estimar cuánto cuesta cada pendiente y decir qué desbloquea
La bitácora: leerla después de cada entrega. Es el único documento que la escuela tiene que seguir leyendo	Escribir en la bitácora cada decisión y su porqué, en cada sesión

4. El proceso, paso a paso

Esta es la parte que se acuerda entre los dos.

4.1 La reunión de entrega (una hora)

Con la computadora de la escuela, el sistema corriendo y los PDF a la mano.

MINUTOS	QUÉ	QUIÉN HABLA
0–10	Qué es Lisart y qué problema resuelve. Enseñar `lisart ui` con datos reales: el grafo, una ficha, los roles	La escuela
10–20	Los cuatro proyectos que quiere la escuela (bitácora, sección 2) y en cuál está parado el trabajo: la proyección de pago	La escuela
20–35	`mapa_del_codigo.pdf`, secciones 1, 4, 8 y 12. Los siete niveles, dónde está la línea entre front y back, por dónde empezar	Los dos, con el PDF abierto
35–45	Las reglas que no se negocian (sección 6 de esta guía). Leerlas en voz alta, las siete	La escuela
45–55	Sección 3 de esta guía: quién se queda con qué. Y la sección 4.4: qué quiere decir "terminado"	Los dos
55–60	Acordar el primer encargo (sección 8) y el ritmo (sección 4.6)	Los dos

Lo que el desarrollador se lleva de la reunión: acceso al repositorio con historial, la lista de pendientes, y un encargo concreto con fecha.

4.2 La primera semana del desarrollador

Sin datos reales. El objetivo no es producir, es **poder producir sin romper**.

DÍA	QUÉ	CÓMO SE SABE QUE QUEDÓ
1	Instalar (`README.md`, sección de instalación) y correr `make check`	`make check` termina sin errores en su máquina
1	Leer la bitácora completa y `CLAUDE.md`	Puede decir, sin mirar, por qué la base son CSV y por qué no hay tabla de familias
2	Leer `tables.py`, `io.py`, `schema.py` (mapa, sección 8, pasos 1 y 2)	Puede explicar qué pasa si dos programas escriben el mismo CSV al mismo tiempo, y por qué aquí no pasa
2	Correr cada comando de `lisart` con la base vacía y con `lisart demo`	Abrió las dos pantallas y las vio funcionar

DÍA	QUÉ	CÓMO SE SABE QUE QUEDÓ
3	Leer `procesos/roles.py` de punta a punta	Puede señalar qué parte es dominio y qué parte es pantalla
3–4	**Primer cambio propio**, chico, en `cli.py` o en un proceso del nivel 3	`make check` verde, y una nota de una línea en la bitácora
5	Leer `guia_de_uso.pdf` y hacer, él mismo, la rutina semanal con `--desde-raw` sobre datos de prueba	Puede operar el sistema como lo opera la administración
5	Llamada de media hora: el desarrollador cuenta qué entendió y qué no	La escuela decide si ya se entregan los datos reales

4.3 Cómo se pide un cambio

Un encargo se escribe, no se platica. Cuatro renglones bastan, y el desarrollador no debería empezar sin los cuatro:

```
Qué quiero que pase:    Que la proyección incluya a los 17 alumnos de
                        "Individual (1xsemana)".
Para quién es:          Para quien arma la cobranza del mes.
Cómo es hoy:            Salen listados aparte y el total dice que faltan.
Cómo sabré que quedó:   El total cubre a los 115 y el CSV de "sin precio"
                        sale vacío.
```

El último renglón es el más importante: es lo que los dos van a revisar al final, y evita el "yo pensé que era otra cosa". Si la escuela no sabe cómo llenarlo, esa es la primera pregunta para el desarrollador, antes de que programe nada.

Si el cambio cae en el back-end (sección 2), el desarrollador contesta antes de empezar con **qué datos existentes toca** y **qué pasa si sale mal**. Si la respuesta incluye la palabra *migración*, se agenda con tiempo: no es un cambio de código, es cambiar la forma de datos que ya están cargados.

4.4 Qué quiere decir "terminado"

Un cambio está terminado cuando cumple **todo** esto. No es negociable por partes, y está escrito así en `CLAUDE.md`:

1. `make check` pasa (revisión de estilo, pruebas, documentación al día, datos válidos).
2. Si el cambio **se ve**, se miró: una captura de pantalla, no solo la prueba. Varios defectos de este proyecto solo salieron mirando.
3. Si cambia **cómo se usa** el sistema, está en `docs/guia_de_uso.md` y se regeneró el PDF.
4. Si cambia **qué es** el sistema (una tabla nueva, un proceso nuevo, un pendiente que se cerró), está en `docs/documentacion_general.md`.

5. Si cambia la **estructura del código** (un módulo nuevo, lógica de datos que se mueve fuera de `ui/`), está en `docs/mapa_del_codigo.md`.
6. La **bitácora** tiene la decisión y su porqué, con la fecha del encabezado actualizada.
7. Está en el historial (**commit**) con un mensaje que dice qué cambió.

Los puntos 3 a 5 no son burocracia: son la razón por la que el sistema se pudo entregar. Sin ellos, en tres meses la escuela tiene un PDF impreso que ya no dice la verdad, y ese es el que alguien va a seguir.

4.5 Cómo se entrega de vuelta

Al cerrar un encargo, el desarrollador manda un mensaje corto con cinco cosas. La escuela no tiene que leer código para revisarlo:

```
Qué cambió:        (una o dos frases, en lenguaje de la escuela)
Qué NO cambió:     (lo que se pidió y quedó fuera, y por qué)
Cómo se verificó:  make check verde + captura / corrida sobre datos reales
Qué documento se actualizó:  guía / general / mapa / bitácora
Qué queda abierto: (lo que salió a la luz y no se resolvió)
```

Y la escuela hace una sola cosa: **abre la bitácora y lee la entrada nueva.** Si la entrada explica el porqué y no solo el qué, el trabajo está bien hecho. Si dice "se arregló el bug" sin más, se devuelve.

4.6 El ritmo

- **Una vez por semana**, una llamada de treinta minutos con tres puntos: qué se cerró, qué se abrió, qué se decide. La escuela llega con las decisiones de negocio pendientes; el desarrollador, con sus preguntas.
- **La ingesta semanal de Jotform** (`lisart ingest jotform`) sigue siendo de la administración, no del desarrollador, hasta que quede agendada sola (pendiente E2).
- **Cada cambio en la lista de pendientes** de la bitácora lo escribe el desarrollador, y la prioridad la pone la escuela.

5. Cómo revisar el trabajo sin leer código

Preguntas que la escuela puede hacer en cada entrega, y qué respuesta esperar. Ninguna requiere abrir un archivo `.py`:

PREGUNTA	RESPUESTA BUENA	SEÑAL DE ALARMA
"¿Corriste `make check`?"	"Sí, aquí está la salida, todo en verde"	"Casi todo pasa", "solo falla una prueba vieja"
"¿Qué respaldo se hizo antes de tocar datos?"	"Ninguno hizo falta, no toqué `master/`" o "está en `data/backups/<tabla>/`"	"Lo hice directo, era rápido"
"¿Esto es de cómo se ve o de qué es verdad?"	Contesta sin dudar, y si es de lo segundo dice qué datos tocó	No sabe distinguir
"¿Qué documento actualizaste?"	Nombra uno, y el PDF está regenerado	"Luego lo documento"
"¿Qué dice la bitácora de esto?"	Hay entrada, con fecha y porqué	No hay entrada, o dice solo qué se hizo
"¿Qué pasa si esto falla a media escritura?"	"Nada, `io.py` escribe a un archivo temporal y lo cambia al final"	Si no sabe contestarlo, no ha leído `io.py`
"¿Puedo abrir el CSV en Excel y ver el cambio?"	Sí, y te enseña cuál	Si el dato solo se ve en la pantalla, algo se guardó fuera de `master/`

Y una prueba que cualquiera puede hacer: **abrir `lisart ui`, buscar a una persona conocida y ver que sigue ahí con sus datos completos.** Si un cambio de back-end salió mal, este es el lugar donde se nota primero.

6. Las reglas que no se negocian

Están en `CLAUDE.md` con su explicación técnica. Aquí van en lenguaje de la escuela, con **cómo saber si se respetaron** sin leer código:

REGLA	POR QUÉ EXISTE	CÓMO SE COMPRUEBA
Nadie escribe en `data/` más que `io.py`	Ahí viven los tipos, la validación y la garantía de no dejar un archivo a medias. Saltárselo es como escribir en el libro de cuentas con lápiz distinto	`lisart validate` no reporta errores después de cada cambio
Las tablas se definen en un solo archivo (`tables.py`)	De ahí salen la pantalla, la validación y la documentación. Definirlas en dos lugares es tener dos verdades	Un cambio de tabla trae consigo `modelo_de_datos.md` regenerado
Los catálogos no se llenan desde el código	Si el formulario trae "Piano " con espacio, darlo de alta solo llena el catálogo de variantes mal escritas	El sistema **avisa** de valores desconocidos; nunca aparece un instrumento que nadie escribió
Nada se sobrescribe sin respaldo	Un error de captura se deshace; un archivo pisado no	`data/backups/<tabla>/` tiene una copia de antes de cada guardado
Un rol no se borra: se cierra (y así irán los pagos)	Borrar hace imposible entender por qué una clase de marzo tenía a alguien que "nunca fue profesor"	Las filas viejas siguen ahí con `vigente_hasta` lleno
Los documentos van con su PDF, y la bitácora al día	El PDF es lo que la escuela lee; un `.md` cambiado sin PDF es un documento que miente	`make check` truena si un PDF quedó atrás
`data/` no sale de la computadora de la escuela	Son datos de menores	Nunca hay un CSV de `master/` en un correo, un chat ni una carpeta sincronizada

7. Lo que el desarrollador va a querer cambiar, y por qué no (todavía)

Cada una de estas fue una decisión, no un descuido. El desarrollador puede proponer cambiarlas —con argumentos y en la llamada semanal—, pero no hacerlo por su cuenta:

LO QUE VA A QUERER	POR QUÉ ESTÁ ASÍ	DÓNDE ESTÁ EXPLICADO
Cambiar los CSV por una base de datos de verdad (SQLite, Postgres)	A escala de cientos de personas, un CSV disciplinado se abre en Excel, se respalda copiando una carpeta y no cobra renta. La disciplina la pone `io.py`	Bitácora, sección 1; `documentacion_general`, sección 2
Usar `pd.read_csv` directo "porque es más rápido"	Se salta tipos, validación y escritura atómica. Es la regla número uno	`CLAUDE.md`; `convenciones_datos.md`
Instalar plotly o altair para las gráficas	No dibujan nada en este equipo; está comprobado con una app mínima	`CLAUDE.md`, "Trampas del entorno"
Normalizar nombres al ingestar ("DE LA O" → "De La O")	En `interim` el texto se guarda como llegó; normalizar es trabajo del alta a `personas`, y ahí se hace con cuidado	Bitácora, sesión 3
Guardar las familias en una tabla	Se deducen del tutor común. Guardarlas es materializar una inferencia que se desactualiza sola	Bitácora, sesión 4
Reemplazar el pentagrama y los iconos SVG por algo "estándar"	Son la identidad del sistema y la única forma de dibujar que funciona aquí	Bitácora, sesión 6
Resolver una fecha de nacimiento en conflicto con "la más reciente"	Los tutores capturan la fecha del hijo. Gana la autodeclarada; si no se puede, queda vacía	Bitácora, sesión 4
Usar `pandera` u otra librería de validación	Las reglas son simples y ninguna hace llaves foráneas entre tablas. Si hace falta, el cambio va solo en `validate.py`	Bitácora, sección 4

Y una trampa que no es decisión sino descubrimiento: **`if fila["x"]` con una celda vacía de pandas revienta** ("boolean value of NA is ambiguous"). Ya pasó más de una vez en este proyecto, en archivos distintos. Se usa `tiene_valor()` o `pd.isna()`.

8. El primer encargo, y los que siguen

La lista completa y ordenada está en la bitácora, sección 5. Para elegir el primero, el criterio es que sea **visible, chico y que no toque datos**. Candidatos, en orden:

ENCARGO	POR QUÉ ES BUEN PRIMERO	DÓNDE CAE
E2 — Agendar la ingesta semanal (cron)	El comando existe; falta que corra solo. Se ve el resultado en una semana y no toca código de datos	Andamio
F6 — Quitar una clase o desasignar un profesor en la app de profesores	Un cambio completo de punta a punta: revisar referencias, cerrar en vez de borrar, pantalla, prueba	Proceso + pantalla
B1b — Precio de "Individual (1xsemana)"	No es código: es un renglón en el Excel de la escuela. Pero el desarrollador acompaña la ingesta y ve el ciclo completo	Datos, con la escuela
F2 — Crear y borrar registros desde la pantalla	Más grande. Necesita generar IDs y revisar referencias. Segundo mes, no primero	Back-end dentro de la pantalla

Lo que **no** conviene como primer encargo: B2 (la tabla de pagos) y C1 (el conector de Kydemy). Son los más valiosos y justo por eso necesitan que quien los haga ya conozca `io.py`, `ids.py` e `identificadores_externos`. Van después de la primera entrega limpia.

9. Palabras que se usan aquí

PALABRA	QUÉ QUIERE DECIR
Delegar	Pasar el mantenimiento del código a otra persona. No incluye las llaves, los datos ni las decisiones de negocio
Back-end	La parte del sistema que decide qué es verdad: datos, reglas, cálculos
Front-end	La parte que decide cómo se ve: pantallas, dibujos, lo que se imprime
Repositorio / repo	La carpeta del proyecto con su historial de cambios
`git` **/ historial**	El registro de cada cambio: quién, cuándo y qué. Permite ver y deshacer
Commit	Un cambio guardado en el historial, con un mensaje que lo describe
`make check`	La revisión completa. Si pasa, el cambio se puede entregar
Bitácora	`.claude/estado-del-proyecto.md`. Las decisiones con su porqué, y los pendientes. El único documento que la escuela sigue leyendo
Encargo	Un cambio pedido por escrito con los cuatro renglones de la sección 4.3
Terminado	Que cumple los siete puntos de la sección 4.4. No hay "casi terminado"
Migración	Cambiar la forma de datos que ya existen. Nunca es solo código, y se agenda aparte
`.env`	El archivo con las llaves. No se manda por chat ni entra al historial
`LISART_DATA_DIR`	La forma de tener los datos en otra carpeta, fuera del código
Catálogo	Lista cerrada que mantiene la administración: espacios, instrumentos, precios, descuentos
Respaldo	Copia del archivo antes de cada guardado, en `data/backups/`